

Java LinkedList — Student Notes

Clear explanations, examples, and dry runs for revision.

1. What is a LinkedList?

A `LinkedList` stores elements as separate **nodes** connected by references, instead of in one continuous block of memory like an array. Each node holds its data plus a link to the *next* node and, in Java, also a link to the *previous* node — this is called a **doubly linked list**. It lives in `java.util` and implements both `List` and `Deque`.

Because there's no fixed array, a `LinkedList` never needs to "resize" the way an `ArrayList` does. Inserting or deleting anywhere just means re-pointing a couple of links.

```
List<Integer> numbers = new LinkedList<>();
numbers.add(10);
numbers.add(20);
numbers.add(30);
System.out.println(numbers); // [10, 20, 30]
```

Step	Statement	State after
1	<code>new LinkedList<>()</code>	<code>[]</code> empty
2	<code>add(10)</code>	<code>[10]</code>
3	<code>add(20)</code>	<code>[10, 20]</code>
4	<code>add(30)</code>	<code>[10, 20, 30]</code>

2. The Node

Every element is wrapped in a `Node` object:

```
class Node<E> {
    E item;
    Node<E> next;
    Node<E> prev;
}
```

The list itself only keeps two references: `first` (head) and `last` (tail). To reach any other node, you must travel link by link from one of these two ends — there's no shortcut like array indexing.

3. LinkedList vs ArrayList

Feature	ArrayList	LinkedList
Structure	Resizable array	Chain of nodes
<code>get(i)</code>	$O(1)$	$O(n)$
Insert/remove at start	$O(n)$	$O(1)$
Insert/remove at end	Amortized $O(1)$	$O(1)$
Insert/remove in middle	$O(n)$	$O(n)$ to find + $O(1)$ to link
Memory per element	Low	Higher (2 extra references)
Implements	<code>List</code>	<code>List</code> , <code>Deque</code>

Rule of thumb: choose `ArrayList` when you read a lot by index; choose `LinkedList` when you insert/delete a lot at the ends or via an iterator.

4. Singly vs Doubly Linked

- **Singly linked** — each node has only `next`. Travel forward only.
- **Doubly linked** — each node has both `next` and `prev`. Travel either direction; Java's built-in `LinkedList` is always this kind.

5. Declaring and Creating

```
List<String> names = new LinkedList<>();
Deque<String> deque = new LinkedList<>();
List<String> copy = new LinkedList<>(names); // independent copy
```

There is no capacity constructor (unlike ArrayList's `new ArrayList<>(50)`) because there is no backing array to size up front.

6. How Insertion Actually Works

addLast(x): create a new node whose `prev` points at the old last node; make it the new `last`; tell the old last node its `next` is this new node.

addFirst(x): the mirror image at the front — both are $O(1)$, no shifting, no copying.

Step	Call	first	last
1	addLast(10)	10	10
2	addLast(20)	10	20
3	addLast(30)	10	30

This is why `LinkedList.addFirst(x)` is $O(1)$, but `ArrayList.add(0, x)` is $O(n)$ — the ArrayList must shift every existing element one slot right.

7. Core Methods

Method	Purpose	Cost
add(x) / addLast(x)	append to end	$O(1)$
addFirst(x)	insert at front	$O(1)$
add(i, x)	insert at index	$O(n)$
get(i)	read by index	$O(n)$
getFirst() / getLast()	peek at ends	$O(1)$
set(i, x)	overwrite at index	$O(n)$
remove(i) / removeFirst() / removeLast()	remove by index or end	$O(n)$ / $O(1)$ / $O(1)$
contains(x)	membership check	$O(n)$
push(x) / pop()	stack (LIFO)	$O(1)$
offer(x) / poll()	queue (FIFO)	$O(1)$

remove(int) vs remove(Object) — the classic trap

```
List<Integer> list = new LinkedList<>(List.of(10, 20, 30));
list.remove(1);           // removes INDEX 1 -> [10, 30]
list.remove(Integer.valueOf(10)); // removes VALUE 10 -> [30]
```

An `int` argument always picks the index overload. To remove by value, wrap it with `Integer.valueOf(x)` so Java picks the Object overload instead.

8. Iterating

```
// for-each: simplest, read-only
for (int v : list) { System.out.println(v); }

// ListIterator: can move both directions and safely add/remove mid-loop
ListIterator<Integer> it = list.listIterator();
while (it.hasNext()) {
    int v = it.next();
    if (v % 2 == 0) it.remove();
}

// descendingIterator: walks from last to first
Iterator<Integer> dit = list.descendingIterator();
```

Never use `for (int i = 0; i < list.size(); i++) list.get(i)` on a LinkedList — each `get(i)` re-walks the chain, making the whole loop $O(n^2)$ instead of $O(n)$. Use for-each or an iterator instead.

v	even?	Action	list after
10	yes	remove	[15, 22, 7, 8]
15	no	keep	[15, 22, 7, 8]
22	yes	remove	[15, 7, 8]
7	no	keep	[15, 7, 8]
8	yes	remove	[15, 7]

9. LinkedList as Stack / Queue (Deque)

Role	Add	Remove	Order
Stack	push(x)	pop()	LIFO
Queue	offer(x)	poll()	FIFO
Deque	addFirst / addLast	removeFirst / removeLast	either end

```
Deque<Integer> stack = new LinkedList<>();
stack.push(1); stack.push(2); stack.push(3);
stack.pop(); // 3 (last pushed, first out)

Deque<Integer> queue = new LinkedList<>();
queue.offer(1); queue.offer(2); queue.offer(3);
queue.poll(); // 1 (first offered, first out)
```

push/pop work at the front internally (both $O(1)$), which is why LinkedList makes a good stack — unlike ArrayList, where popping from index 0 would be $O(n)$.

10. Searching

Only linear search is practical — start at `first` (or `last`) and walk node by node comparing values, using `indexOf(x)` or `contains(x)`. Binary search doesn't help here because reaching the "middle" node still requires walking through every node before it.

index	value	match target 7?
0	4	no
1	9	no
2	2	no
3	7	yes → return 3

11. Sorting

```
List<Integer> nums = new LinkedList<>(List.of(5, 3, 8, 1, 9));
Collections.sort(nums); // [1, 3, 5, 8, 9]
nums.sort(Comparator.reverseOrder()); // [9, 8, 5, 3, 1]
```

Java copies the elements to an array, sorts with TimSort ($O(n \log n)$), then relinks the nodes in the new order.

12. Custom Objects

```
class Student {
    String name; int marks;
    Student(String name, int marks) { this.name = name; this.marks = marks; }
    public String toString() { return name + ":" + marks; }
}

LinkedList<Student> students = new LinkedList<>();
students.addLast(new Student("Riya", 88));
students.addFirst(new Student("Aman", 72));
System.out.println(students); // [Aman:72, Riya:88]
```

13. Building a Simple Singly Linked List

```

class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; }
}
class SimpleLinkedList {
    Node head;
    void addLast(int value) {
        Node newNode = new Node(value);
        if (head == null) { head = newNode; return; }
        Node curr = head;
        while (curr.next != null) curr = curr.next;
        curr.next = newNode;
    }
}

```

This version has no tracked tail, so `addLast` must walk the whole list every time ($O(n)$) — adding a tail reference would make it $O(1)$, matching how Java's real `LinkedList` works.

14. Time Complexity Cheat-Sheet

Operation	Time
addFirst / addLast	$O(1)$
removeFirst / removeLast	$O(1)$
get(i) / set(i, x)	$O(n)$
add(i, x) / remove(i)	$O(n)$
contains(x) / indexOf(x)	$O(n)$
size()	$O(1)$
push/pop/offer/poll	$O(1)$
sort	$O(n \log n)$

15. Common Mistakes to Avoid

- Using `get(i)` in a loop $\rightarrow O(n^2)$. Use for-each or an iterator.
- Mixing up `remove(int)` and `remove(Object)`.
- Forgetting `LinkedList` already works as a stack/queue via `push/pop/offer/poll`.
- Assuming `LinkedList` is always faster — it's only better for insert/delete at known positions, not for reading.
- Modifying a list with for-each instead of using an iterator's own `add/remove`.